Docker Compose Experiments Guide

Over

view

This guide covers two comprehensive experiments on Docker containerization:

-
- Experiment 6A: Comparison of Docker Run and Docker Compose
Experiment 6B: Multi-Container Application using Docker Compose (WordPress + Database)

Table of Contents

Experiment 6A: Docker Run vs Docker Compose

Objective

Understand the relationship between `docker run` and Docker Compose, compare their configuration syntax, and learn their use cases.

Background Theory

2.1 Docker Run (Imperative Approach)

The docker run command creates and starts a container from an image with explicit flags:

- Port mapping (-p)
- Volume mounting (-v)
- Environment variables (-e)
- Network configuration (–network)

- Restart policies (–restart)
- Resource limits (–memory, –cpus)
- Container name (–name)

Example:

```
docker run -d \
 --name my-nginx \
 -p 8080:80 \
 -v ./html:/usr/share/nginx/html \
 -e NGINX_HOST=localhost \  --
restart unless-stopped \
nginx:alpine
```

2.2 Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes in a structured format.

Equivalent Compose File:

```
version: '3.8'

services:
nginx:
    image: nginx:alpine
    container_name: my-
nginx     ports:      -
"8080:80"       volumes:
    - ./html:/usr/share/nginx/html
    environment:
    NGINX_HOST: localhost
    restart: unless-stopped
```

Mapping: Docker Run vs Docker Compose

Docker Run Flag	Docker Compose Equivalent
`-p 8080:80`	`ports:`
`-v host:container`	`volumes:`
`-e KEY=value`	`environment:`
`--name`	`container_name:`
`--network`	`networks:`

Docker Run Flag	Docker Compose Equivalent
`--restart`	`restart:`
`--memory`	`deploy.resources.limits.memory`
`--cpus`	`deploy.resources.limits.cpus`
`-d`	`docker compose up -d`

Advantages of Docker Compose

✓ Simplifies multi-container applications
✓ Provides reproducibility
✓ Version controllable configuration
✓ Unified lifecycle management
✓ Supports service scaling

```
docker compose up --scale web=3
```

Practical Tasks (Experiment 6A)

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

```
docker run -d \  --
name lab-nginx \
 -p 8081:80 \
 -v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine

# Verify
docker ps

# Access
# http://localhost:8081

# Stop and remove
docker stop lab-nginx
docker rm lab-nginx
```

Step 2: Run Same Setup Using Docker Compose

```
version: '3.8'
```

```
services:
nginx:
    image: nginx:alpine
    container_name: lab-
nginx     ports:      -
"8081:80"       volumes:
     - ./html:/usr/share/nginx/html
```

```
# Run docker
compose up -d

# Verify docker
compose ps

# Stop docker
compose down
```

Task 2: Multi-Container Application (WordPress + MySQL)

A. Using Docker Run

```
# Create network docker
network create wp-net

# Run MySQL
docker run -d \
 --name mysql \
 --network wp-net \
 -e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7

# Run WordPress
docker run -d \
 --name wordpress \
 --network wp-net \
 -p 8082:80 \
 -e WORDPRESS_DB_HOST=mysql \  -
e WORDPRESS_DB_PASSWORD=secret \
wordpress:latest

# Test
# http://localhost:8082
```

B. Using Docker Compose

```
version: '3.8'

services:
mysql:
    image: mysql:5.7
    environment:
```

```
    MYSQL_ROOT_PASSWORD: secret
    MYSQL_DATABASE: wordpress
    volumes:
    -     mysql_data:/var/lib/mysql

  wordpress: image:
    wordpress:latest
    ports:
    -     "8082:80" environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD:
    secret depends_on: - mysql

volumes:
mysql_data:
```

```
docker compose up -d
docker compose down -v
```

Task 3: Convert Docker Run to Docker Compose

Problem 1: Basic Web Application

Given Docker Run Command:

```
docker run -d \  --
name webapp \
 -p 5000:5000 \
 -e APP_ENV=production \
 -e DEBUG=false \  --
restart unless-stopped \
node:18-alpine
```

Docker Compose Solution:

```
version: '3.8'


services:
webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
    - "5000:5000"
    environment:
    APP_ENV: production
    DEBUG: "false"
    restart: unless-
stopped
```

Problem 2: Volume + Network Configuration

```
version: '3.8'

services:  postgres-
db:       image:
postgres:15
    networks:   -
app-net
    environment:
    POSTGRES_USER: admin
    POSTGRES_PASSWORD:
secret    volumes:
      -   pgdata:/var/lib/postgresql/data

  backend: image:
    python:3.11-slim
    networks:
      -   app-net ports:
      -   "8000:8000" environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:
      -   postgres-db

networks:  app-
net:

volumes:
pgdata:
```

Task 4: Resource Limits Conversion

Given Docker Run Command:

```
docker run -d \
 --name limited-app \
 -p 9000:9000 \
 --memory="256m" \
 --cpus="0.5" \  --
restart always \
nginx:alpine
```

Docker Compose Solution:

```
version: '3.8'

services:
 limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
```

```
    - "9000:9000"
    restart: always
    deploy:
    resources:
    limits:
    memory: 256m
    cpus: "0.5"
```

Note: The `deploy` section works in Docker Swarm mode. In regular Compose mode, it may be ignored.

Advanced Tasks (Building Custom Images)

Task 5: Replace Standard Image with Dockerfile

Step 1: Create app.js

```
const http = require('http');

http.createServer((req, res) =>
{  res.end("Docker Compose Build Lab");
}).listen(3000);
```

Step 2: Create Dockerfile

```
FROM node:18-alpine
WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step 3: Create docker-compose.yml

```
version: '3.8'

services:
nodeapp:
    build:
    context: .
```

```
    dockerfile: Dockerfile
    container_name: custom-node-
app      ports:      -
"3000:3000"
```

Running the Application

```
# Build and run docker
compose up --build -d

# Verify
# http://localhost:3000

# Rebuild after changes
docker compose up --build -
d
```

Key Difference:

image: - Uses pre-built image
build: - Builds image from Dockerfile

Task 6: Multi-Stage Dockerfile with Compose

Multi-Stage Dockerfile Example:

```
# Stage 2:
FRO python:3.11-
WORKDI
COP --from=builder /usr/local/lib/python3.11/site-packages
COP
EXPOS
CM ["python",
```

Benefits: ✓ Smaller final image
✓ Faster deployments
✓ Better security (fewer dependencies)

```
# Stage 1: Build
FROM python:3.11-slimas builder
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
```

Experiment 6B: WordPress with Docker Compose

Objective

Deploy a multi-container WordPress application with MySQL database, understanding container networking, volumes, and scaling.

Architecture Overview

```
User (Browser)
      ↓
WordPress Container (Port 8080)
      ↓
MySQL Container
      ↓
Persistent Volume (Database Storage)
```

Prerequisites

- Docker installed
- Docker Compose (comes with modern Docker)
- Basic understanding of containers

Complete Setup Guide

Step 1: Create Project Directory

```
mkdir wp-compose-lab
cd wp-compose-lab
```

Step 2: Create docker-compose.yml

```
version: '3.9'

services:
db:
    image: mysql:5.7
    container_name:
wordpress_db    restart:
always    environment:
     MYSQL_ROOT_PASSWORD: rootpass
     MYSQL_DATABASE: wordpress
    MYSQL_USER: wpuser
    MYSQL_PASSWORD: wppass
    volumes:
      -    db_data:/var/lib/mysql

  wordpress:
```

```
    image: wordpress:latest
    container_name:
    wordpress_app depends_on:
      -   db ports: - "8080:80" restart: always environment:
     WORDPRESS_DB_HOST: db:3306
     WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD:
      wppass
      WORDPRESS_DB_NAME:
    wordpress volumes:
      -   wp_data:/var/www/html

volumes:
db_data:
wp_data:
```

Step 3: Start Application

```
docker-compose up -d
```

What happens:

- Images are pulled from Docker Hub
- Docker network is created automatically
- Both containers are started
- DNS-based service discovery is enabled

Step 4: Verify Containers

```
docker ps
```

Expected output:

```
wordpress_app (WordPress)
wordpress_db (MySQL)
```

Step 5: Access WordPress

Open your browser and navigate to:

```
http://localhost:8080
```

Complete the WordPress setup:

1. Select language
2. Enter site title
3. Create admin user
4. Enter admin password
5. Complete installation

Step 6: Check Volumes

```
docker volume ls
```

db_data → Database persistence
wp_data → WordPress files

Step 7: Stop Application

```
docker-compose down
```

Note: Volumes remain intact even after containers are removed

Key Concepts

Explanation of Key Sections

ser vices

Defines all containers that make up the

application: db → MySQL database container

wordpress → WordPress application

container depends_on

Ensures the database starts before WordPress:

```
depends_on:
- db
```

environment

Configures container environment variables for database credentials and connection details.

volumes

Persists data even if containers are deleted or recreated:

db_data → MySQL data directory

wp_data → WordPress installation files

por ts

Exposes services to the host machine:

```
ports:
- "8080:80" # Host:Container
```

Scaling in Docker Compose

Method 1: Scale WordPress Containers

```
docker-compose up --scale wordpress=3
```

Result: 3 WordPress containers running

Problem: All containers try to use the same port (8080) → Port conflict

Solution: Use a reverse proxy (Nginx) for load balancing

Limitations of Compose Scaling

- ❌ No built-in load balancing
- ❌ No auto-healing
- ❌ Single host only
- ❌ Not production-ready for scaling

Docker Swarm: Production Scaling

Step 1: Initialize Swarm

```
docker swarm init
```

Step 2: Deploy Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

Step 3: Scale Service

```
docker service scale wpstack_wordpress=3
```

Comparison: Docker Compose vs Docker Swarm

Feature	DockerCompose	Docker Swarm
Scope	Single host	Multi-node cluster
Scaling	Manual	Built-in
Load Balancing	No	Yes (internal LB)
Self-Healing	No	Yes
Rolling Updates	No	Yes
Networking	Basic	Overlay network

Benefits of Docker Swarm

✓ Built-in load balancing
✓ Automatic container restart (self-healing)
✓ Horizontal scaling across nodes
✓ Rolling updates without downtime
✓ Service abstraction (not individual containers)

Limitations of Docker Swarm

- Less popular than Kubernetes
- Limited ecosystem
- Less flexible scheduling
- Fewer enterprise features

Quick Reference

Common Docker Compose Commands

```
# Start services
docker compose up -d

# View running containers
docker compose ps

# View logs docker compose logs -f
[service_name]

# Execute command in container docker
compose exec [service_name] [command]

# Stop services (keep volumes)
docker compose stop
```

```
# Stop and remove containers (keep volumes)
docker compose down

# Stop and remove everything including
volumes docker compose down -v

# Rebuild images docker
compose up --build -d

# Scale services
docker compose up --scale [service]=[number] -d

#   View   images
docker  images  #
View      volumes
docker volume ls
```

Learning Outcomes

After completing these experiments, you will understand:

1. Imperative vs Declarative: Difference between docker run and Docker Compose
2. Multi-Container Applications: How to orchestrate multiple services
3. Networking: How containers communicate using DNS
4. Persistence: Volume management for data persistence
5. Scaling: Limitations of Compose and advantages of Docker Swarm
6. Custom Images: Building images with Dockerfile and using them in Compose
7. Resource Management: Setting memory and CPU limits
8. Production Deployment: Docker Swarm for production-ready clusters

Conclusion

This comprehensive guide demonstrates:

✓ How to deploy multi-container applications using Docker Compose
✓ How containers communicate using internal networking
✓ The importance of volumes for data persistence
✓ Scaling limitations of Docker Compose
✓ Advantages of Docker Swarm for production-ready deployments
✓ Building custom images with Dockerfile integration

Docker Compose is ideal for:

- Development environments
-

Testing applications
Learning containerization

Docker Swarm is ideal for:

- Simple production clusters
- Easy scaling without Kubernetes complexity

Additional Resources

- Docker Compose Official Documentation
- Docker Swarm Documentation
- Dockerfile Best Practices
- Docker Networking Guide

Last Updated: 2024
Experiment Version: 6A & 6BV